VEDA 심화 실습

VEDA 4기 8회차 차용훈, 신채훈
| 2026.04.20 - 2026.04.22

목차

프로젝트 개요 및 주제 선정

- 평가 기간 및 팀 구성
- 주제 선정 배경
- 게임 개발의 장점
- Qt와 C++ 활용

오픈소스 조사 및 분석

- 9개 프로젝트 검토
- 빌드 가능성 확인
- 최종 오픈소스 선정
- 해체 분석 결과

시연 및 추가 기능

- Before/After 비교
- 멀티플레이 구현
- 로그인 및 랭킹
- 추가 예정 기능

CRUD 기능 구현

- 회원가입 및 로그인
- 비밀번호 변경
- 회원 탈퇴
- 랭킹 검색

협업 및 기술 이슈

- 깃허브 브랜치 활용
- 파일 병합 과정
- 빌드 설정 조정
- 네트워크 기능

결론 및 향후 계획

- 학습 성과
- UI 개선 계획
- 추가 기능 개발
- 코드 품질 향상

프로젝트 주제

네트워크 기반 멀티플레이 체스 게임 프로젝트

주제 선정 이유 (1)

유니크한 주제 선택

- 4가지 주제 중 선택
- 고객/일정/계좌 관리 제외
- 다른 팀과 차별화
- 게임 개발로 결정

게임 개발의 장점

- Qt 기능 활용 가능
- C++ 상속 개념 실습
- 객체지향 프로그래밍
- 실전 경험 축적

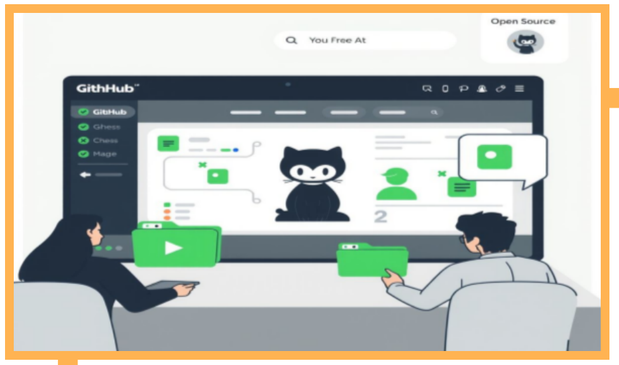
학습 기회 확보

- 예외상황 처리 경험
- 디버깅 능력 향상
- 문제 해결 능력 개발
- 실무 역량 강화

주제 선정 이유 (2)

다양한 예외상황 처리를 통한 학습 기회

- 체스 게임 규칙 구현 시 다양한 예외 케이스 발생
- 폰 프로모션, 캐슬링, 앙파상 등 특수 규칙 처리
- 네트워크 연결 오류 및 동기화 문제 해결 경험



오픈소스 레퍼런스 확보 용이성

- 체스 게임은 오픈소스 프로젝트가 풍부함
- Qt 기반 체스 프로젝트 다수 존재
- 코드 분석 및 학습 자료로 활용 가능

오픈소스 레퍼런스 조사 (1)

PART 01

프로젝트 검토 범위

- 총 9개의 오픈소스 검토
- GitHub 저장소 탐색
- Qt 기반 체스 프로젝트
- 다양한 구현 방식 비교

PART 02

빌드 가능 여부 확인

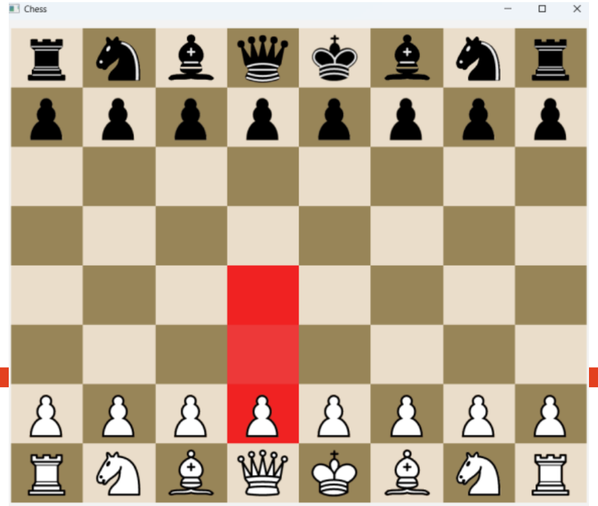
- 각 프로젝트 빌드 테스트
- Qt5/Qt6 호환성 검증
- 컴파일 오류 확인
- 실행 가능성 평가

PART 03

장단점 분석

- 코드 구조 및 가독성
- 기능 구현 완성도
- 파일 분리 및 모듈화
- 확장 가능성 검토

오픈소스 레퍼런스 조사 (2)

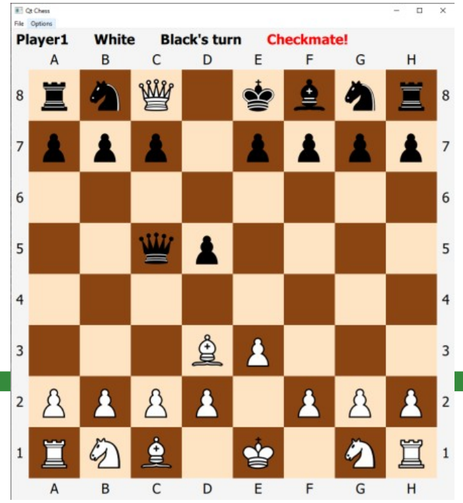


4번 프로젝트

- 각 말의 이동범위 가시화 기능
- AI를 활용한 싱글플레이 구현
- 검은말 이동 표시 개선 필요
- 대기시간 조정 요구사항 존재

5번 프로젝트

- 파일 분리 구조가 우수함
- 각 말의 이동 가능 범위 미표시
- 폰의 퀸 프로모션 시 오류 발생
- 기본 체스 기능은 정상 작동



7번 프로젝트

- Qt5 문법으로 작성되어 Qt6 호환 문제
- 여러 빌드 에러 발생
- 소켓 프로그래밍 부분 참고 가능
- 네트워크 기능 구현 시 활용 가능

최종 오픈소스 선정

PART 01

선정 프로젝트

- 5번 프로젝트 (mdziubich/chessGame_cpp) 최종 선택
- 빌드가 정상적으로 작동하는 프로젝트
- 4번 프로젝트와 비교 검토 후 결정
- 코드 구조의 명확성이 선택의 핵심 요인

PART 02

선정 이유

- 파일 분리가 체계적으로 잘 되어 있음
- 코드의 가독성과 유지보수성이 우수
- 기능 확장 및 수정이 용이한 구조
- 프로젝트 이해와 학습에 적합한 구조

PART 03

외부 리소스 활용

- flaticon에서 체스 말 이미지 리소스 확보
- 외부 리소스 활용 능력 입증
- UI 개선을 위한 고품질 아이콘 사용
- 프로젝트의 시각적 완성도 향상

오픈소스 분석

PART 01

UML 구조 분석

- 프로젝트 내 파일 간 관계 분석
- UML 다이어그램을 통한 구조 파악
- 클래스 간 의존성 및 연관성 확인
- 전체 시스템 아키텍처 이해
- 추가 기능 구현을 위한 구조 분석

PART 02

객체지향 프로그래밍

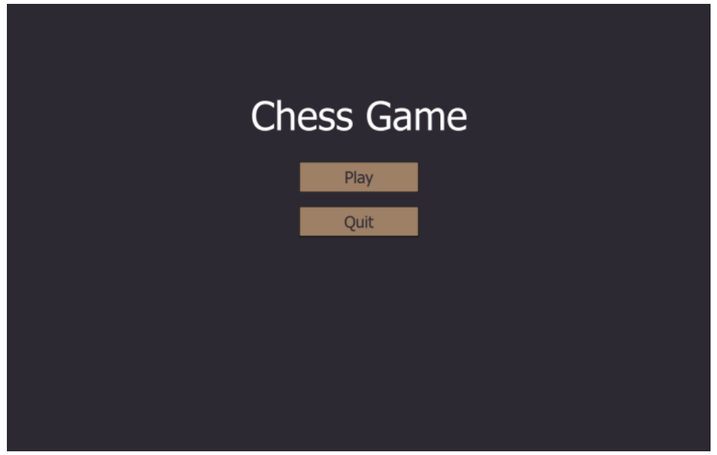
- pawnModel을 기반으로 한 상속 구조
- 객체지향 프로그래밍의 철학 구현
- 코드 재사용성과 확장성 확보
- 다형성을 활용한 유연한 설계
- C++ 상속 개념의 실제 적용 사례

PART 03

상속 구조 활용

- bishoppawnmodel 클래스 구현
- queenpawnmodel 클래스 구현
- 각 체스 말의 특성을 상속으로 표현
- 공통 기능은 부모 클래스에서 관리
- 개별 기능은 자식 클래스에서 구현

시연 모습 - Before (오픈소스 원본)



로컬 플레이 방식

- 단순한 로컬 환경에서만 플레이 가능
- 한 대의 컴퓨터에서 두 명이 번갈아 플레이
- 네트워크 기능 없음
- 기본적인 체스 규칙만 구현됨

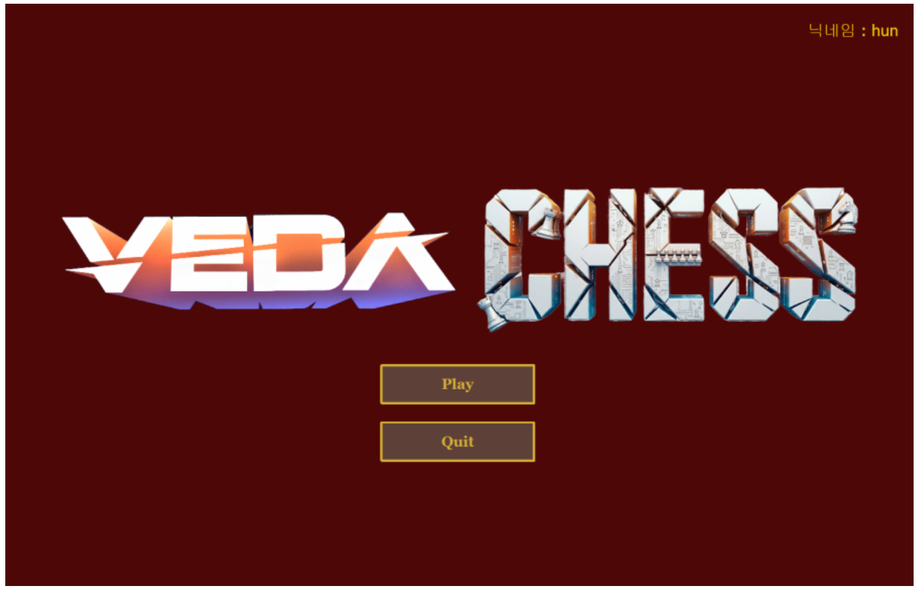
턴 기반 게임 진행

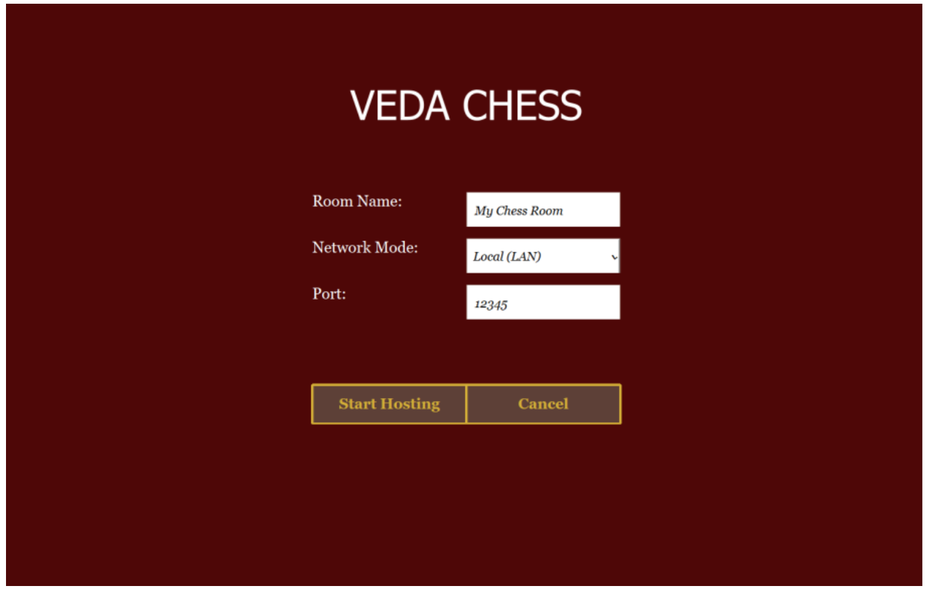
- 한 사람이 말을 두면 다른 사람이 받는 방식
- 같은 화면을 보며 순서대로 진행
- 실시간 대전 기능 부재
- 물리적으로 같은 공간에 있어야 함

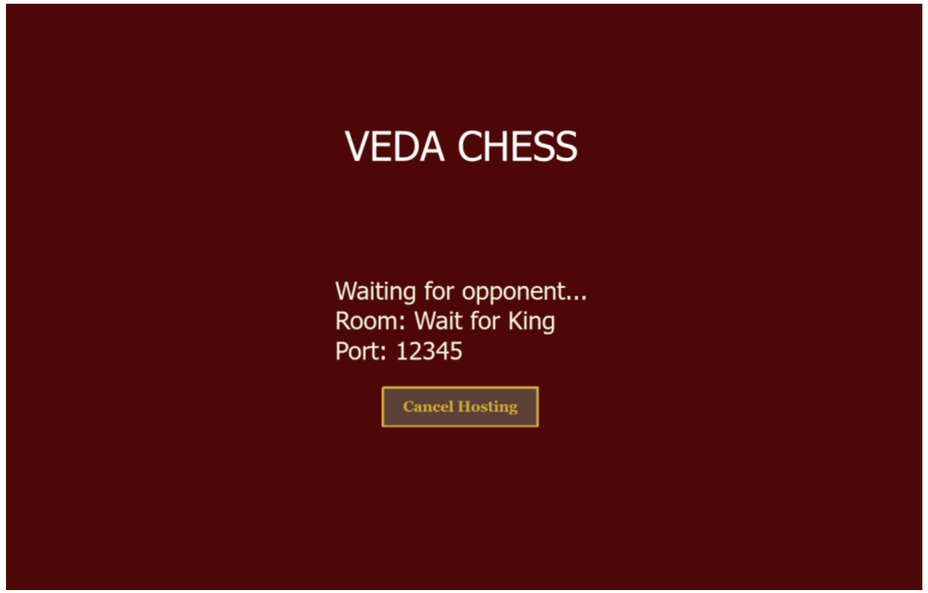
기능적 제약

- 멀티플레이 기능 미구현
- 사용자 인증 시스템 없음
- 전적 기록 기능 없음
- 랭킹 시스템 부재

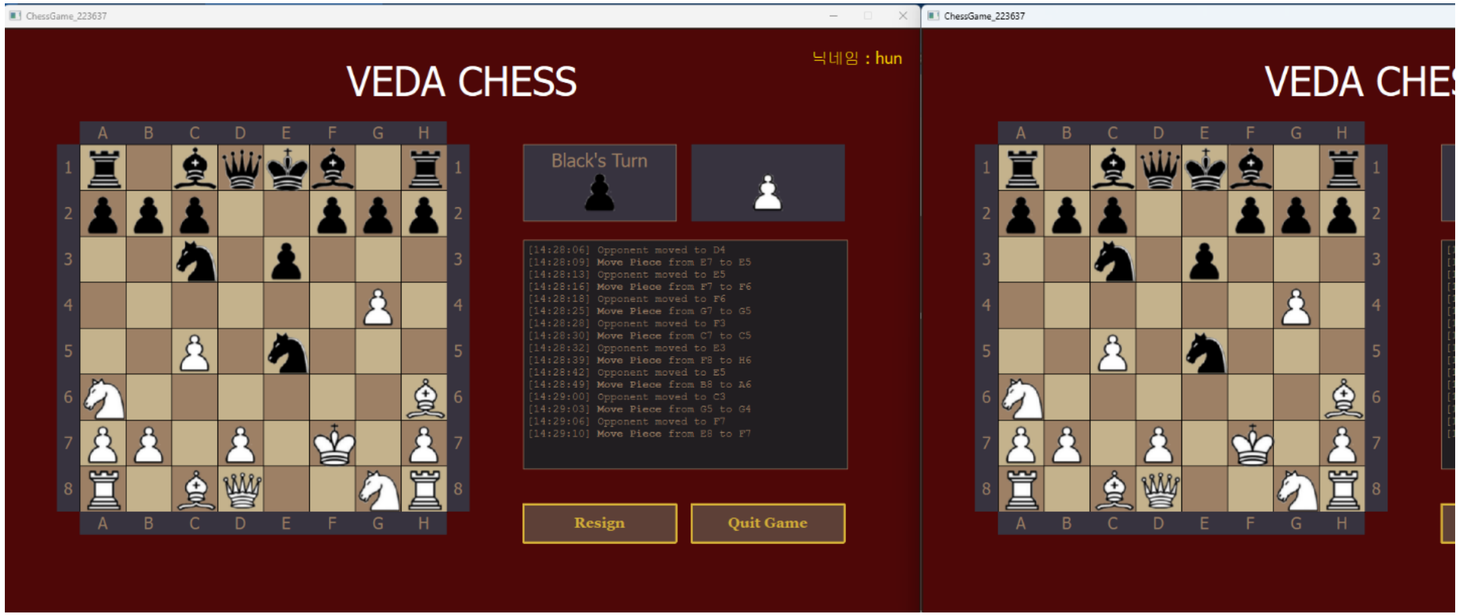
시연 모습 - After (개선된 버전)

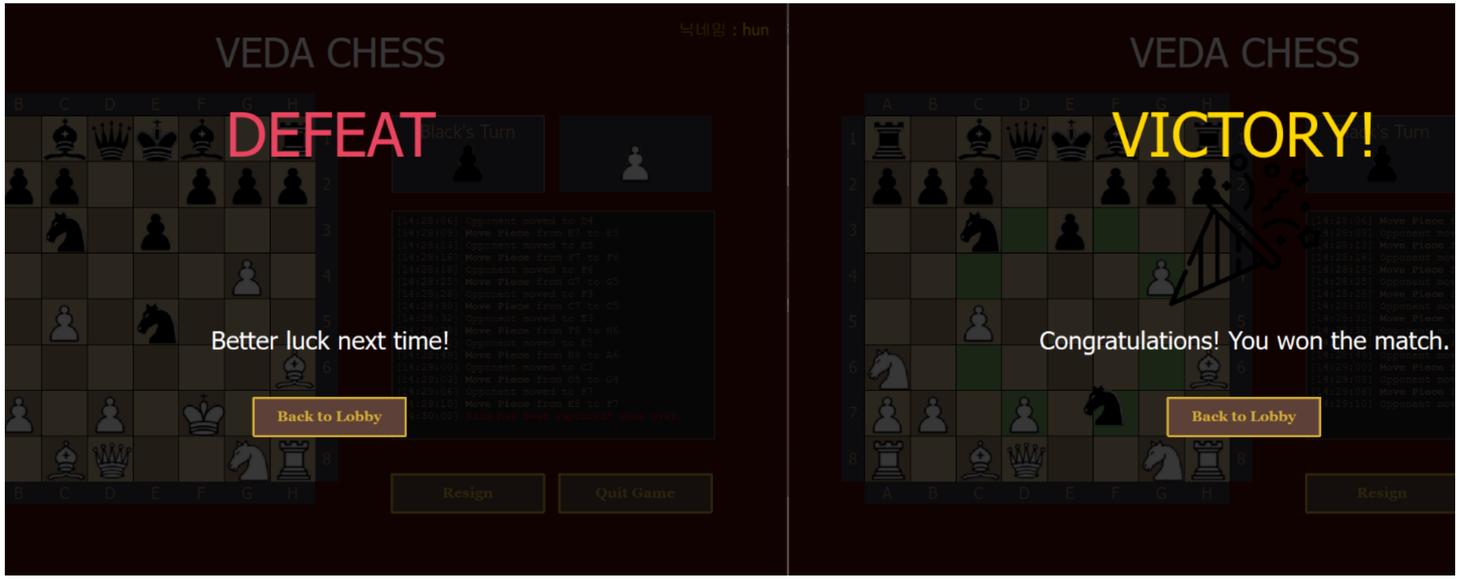




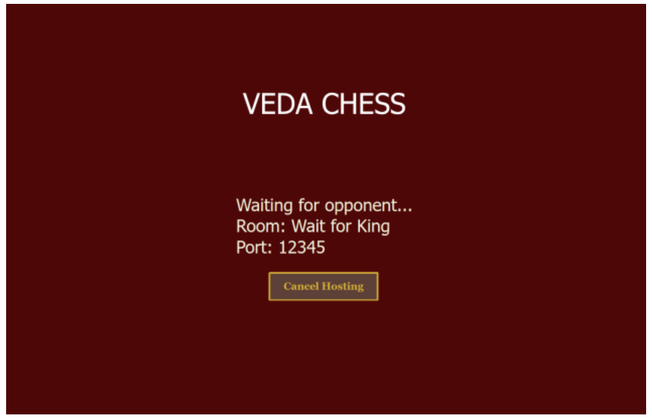


시연 모습 - After (개선된 버전)



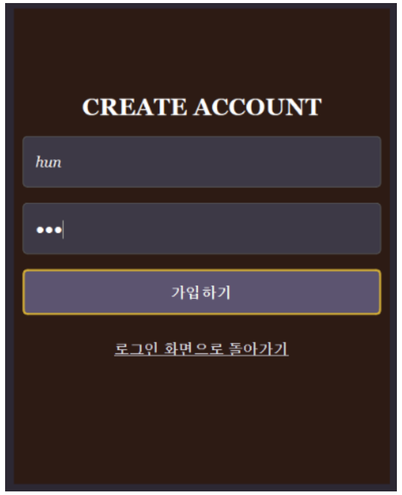


시연 모습 - After (개선된 버전)



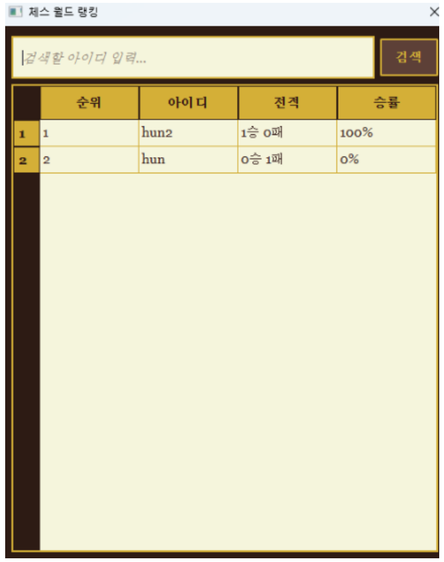
멀티플레이 기능

- 네트워크 기반 멀티플레이 구현
- 서로 다른 PC에서 실시간 대전 가능
- 호스트가 방을 생성하고 클라이언트 참여
- 원격 대전 환경 구축 완료
- 안정적인 네트워크 연결 제공



로그인 시스템

- 회원가입 및 로그인 기능 구현
- 사용자 ID 기반 인증 시스템
- 개인 계정 관리 기능
- 비밀번호 변경 및 회원 탈퇴 지원
- 데이터베이스 기반 사용자 정보 관리



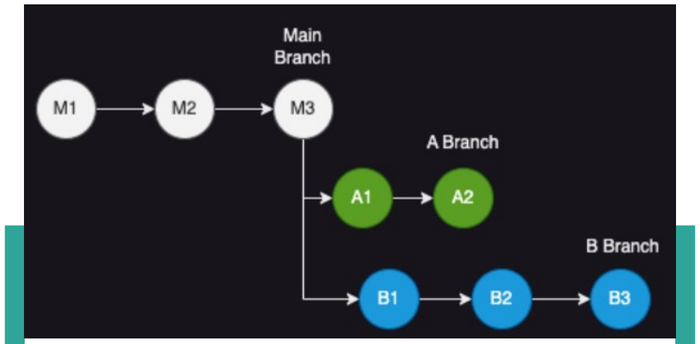
랭킹 시스템

- 게임 전적 기록 시스템 구현
- 승/패 및 승률 통계 제공
- 사용자별 랭킹 검색 기능
- 프로그램 재실행 시 전적 유지
- 경쟁 요소를 통한 게임 재미 향상

깃허브 협업

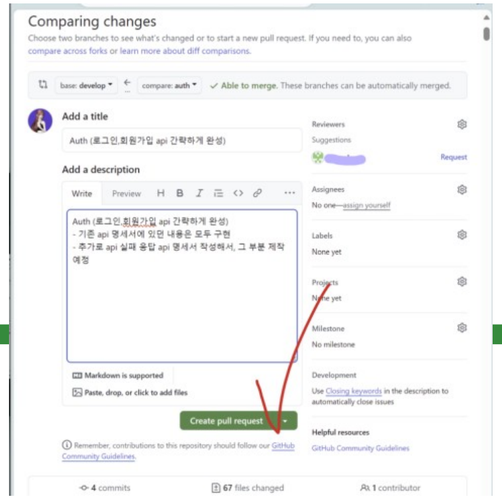
협업을 위한 깃허브 활용

- 팀원 간 코드 공유 및 동시 작업 환경 구축
- 원격 저장소를 통한 프로젝트 관리
- 이슈 트래킹 및 작업 분담



브랜치 기반 협업 분리

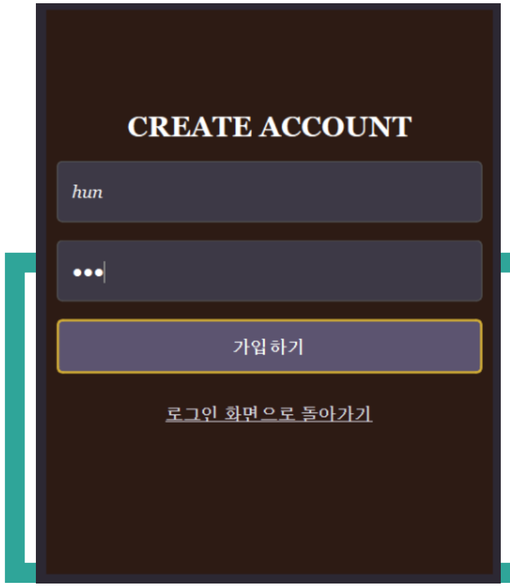
- 기능별 브랜치 생성으로 작업 분리
- 메인 브랜치 보호 및 안정성 유지
- 독립적인 개발 환경 제공



버전 관리 및 코드 통합

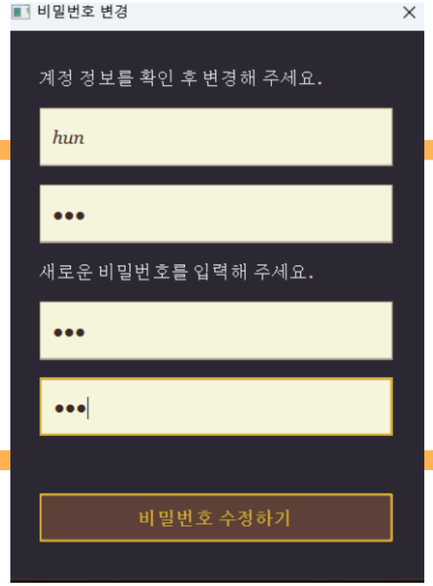
- 커밋 히스토리 관리 및 추적
- 파일 병합(Merge) 및 충돌 해결
- 코드 리뷰 및 품질 관리

CRUD 기능 구현 (1)



데이터 입력(Create): 회원가입 기능

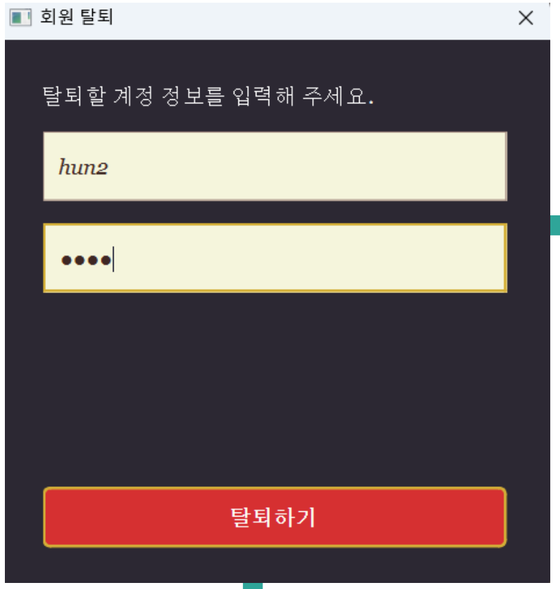
- 사용자 ID 및 비밀번호 입력 폼 구현
- SQLite 데이터베이스에 회원 정보 저장
- 중복 ID 검증 및 유효성 검사 처리



데이터 수정(Update): 비밀번호 변경 기능

- 기존 사용자 인증 후 비밀번호 수정 가능
- 데이터베이스 UPDATE 쿼리 실행
- 변경 사항 즉시 반영 및 확인 메시지 표시

CRUD 기능 구현 (2)

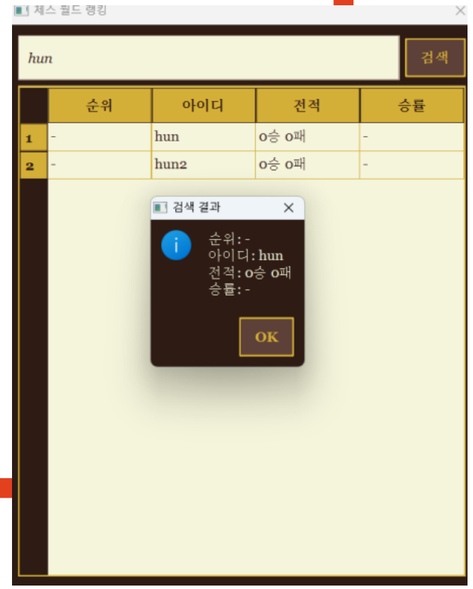


데이터 삭제 (Delete): 회원 탈퇴

- 사용자 계정 삭제 기능 구현
- 데이터베이스에서 회원 정보 완전 제거
- 탈퇴 확인 다이얼로그 표시
- 관련 게임 전적 데이터 처리

- 전체 사용자 랭킹 조회 기능
- 승률 및 전적 기준 정렬
- 데이터베이스 SELECT 쿼리 활용
- 랭킹 테이블 UI 표시

데이터 검색 (Read): 랭킹 검색



UI 개선

PART 01

VEDA 로고 추가

- 로그인 창에 VEDA 로고 배치
- 메인 창 상단에 로고 표시
- 브랜드 아이덴티티 강화
- 시각적 일관성 확보
- 프로젝트 소속감 표현

PART 02

메인창 버튼 중앙 배치

- 메인창 버튼들을 중앙으로 정렬
- 사용자 경험 개선을 위한 레이아웃 조정
- 직관적인 인터페이스 구성
- 버튼 간격 및 크기 최적화
- 깔끔한 UI 디자인 구현

PART 03

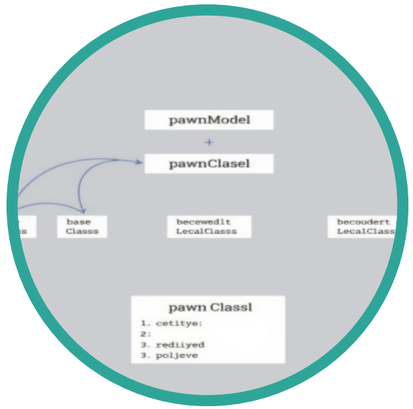
사용자 닉네임 표시 개선

- 메인창에서 사용자 닉네임 표시
- 게임 실행 시 플레이어 정보 가시화
- 로그인 상태 명확히 표시
- 사용자 식별 용이성 향상
- 개인화된 인터페이스 제공

프로젝트를 통한 학습

Qt 프레임워크 활용

- Qt GUI 개발 능력 향상
- 시그널-슬롯 메커니즘 이해
- Qt 위젯 활용 능력 습득
- 크로스 플랫폼 개발 경험



C++ 객체지향 프로그래밍

- pawnModel 상속 구조 학습
- 다형성 개념 실습
- 클래스 설계 및 구현
- 객체지향 설계 원칙 적용

네트워크 프로그래밍

- 소켓 프로그래밍 경험
- 클라이언트-서버 구조 이해
- 멀티플레이 구현 능력
- 네트워크 예외 처리 학습

협업 및 버전 관리

- Git 브랜치 전략 활용
- 코드 병합 및 충돌 해결
- 팀 협업 프로세스 경험
- 버전 관리 능력 개발

오픈소스 분석 및 개선점

코드 비대화 문제

- gameView.cpp 파일이 약 700줄로 증가
- 개발 과정에서 코드가 한 파일에 집중됨
- 기능 추가 시 파일 크기 지속 증가
- 코드 관리의 어려움 발생

리팩토링 필요성

- 지속적인 코드 리팩토링의 중요성 인식
- 기능별 파일 분리 필요성 확인
- 코드 구조 개선을 통한 유지보수성 향상
- 개발 중 주기적인 코드 정리 필요

향후 개선 목표

- 향후 프로젝트에서 좋은 코드 작성 목표
- 초기 설계 단계부터 구조 고려
- 클린 코드 원칙 적용 노력
- 지속적인 코드 품질 개선 추구

추후 네트워크 기능 개선 계획

다른 PC에서 DB 공유 방안

- 여러 PC 간 데이터베이스 공유 필요
- 중앙 서버 구축 방안 검토
- 네트워크를 통한 DB 접근
- 사용자 정보 실시간 공유

서버 연결 또는 파일 공유 방식

- 방법 1: 별도 서버 구축 및 연결
- 방법 2: 파일 공유 방식 활용
- 각 방식의 장단점 비교
- 프로젝트 규모에 맞는 선택

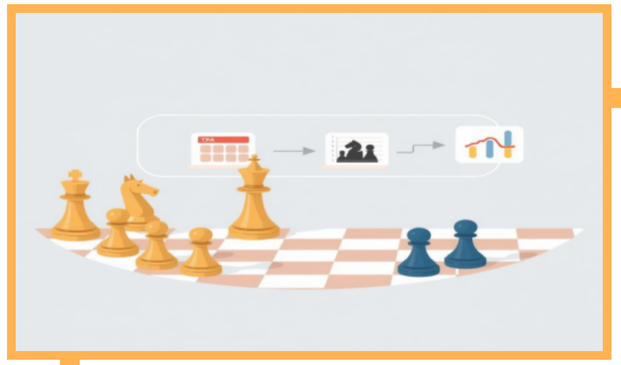
실시간 데이터 동기화

- 게임 결과 즉시 반영
- 랭킹 정보 실시간 업데이트
- 네트워크 지연 최소화
- 데이터 일관성 유지

추가 예정 기능 (1)

체스 퍼즐 기능

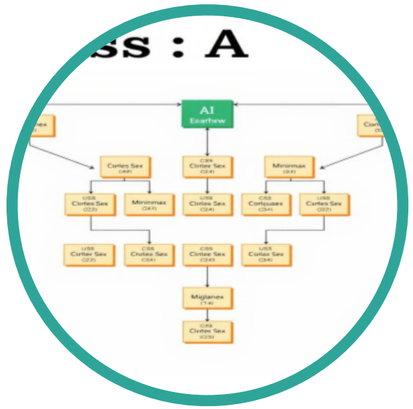
- 체스 전술 훈련을 위한 퍼즐 모드 추가 계획
- 사용자 실력 향상을 위한 학습 콘텐츠 제공
- 난이도별 퍼즐 분류 및 제공

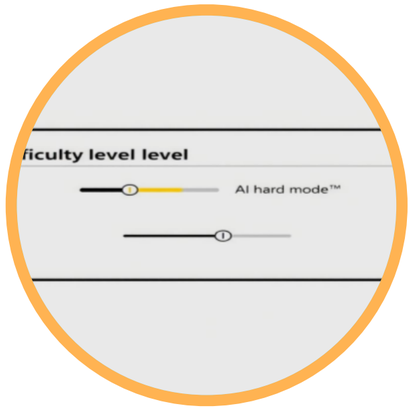


외부 데이터 수집 가능성

- chess.com과 같은 플랫폼에서 퍼즐 데이터 수집 검토
- API 연동 또는 데이터 크롤링 방식 고려
- 저작권 및 라이선스 문제 확인 필요

추가 예정 기능 (2)





AI 싱글 플레이

- 컴퓨터 상대 1인 플레이 모드
- AI 엔진 통합 계획
- 싱글 플레이 수요 충족

오픈소스 참고

- 4번 오픈소스 프로젝트 분석
- AI 구현 방식 학습
- 코드 구조 참고

알고리즘 방식

- 3-5수 앞 경우의 수 계산
- Minimax 알고리즘 활용
- Alpha-beta pruning 적용

난이도 조절

- 다양한 난이도 레벨 제공
- 계산 깊이 조절 방식
- 구현 시간 추가 필요

결론 및 향후 계획

본 프로젝트에서 Qt 기반 멀티플레이 체스 개발을 완료했습니다. 오픈소스 분석, 회원가입·로그인·랭킹 등 CRUD와 네트워크 기반 실시간 멀티플레이를 구현했습니다.

프로젝트를 통해 C++ 객체지향 상속 구조와 Qt 기능 활용, Git 협업을 경험했고, gameView.cpp의 비대화 문제로 리팩토링 필요성을 학습했습니다.

향후 체스 퍼즐, AI 싱글플레이, UI/UX 개선 등을 추가해 완성도 높은 게임으로 발전시킬 계획입니다.